won't replace the backing value. You can bind to an `IEnumerable<>` if you leave its initial value `null`.

Similarly, the binder creates a `Dictionary<,>` as the backing field for properties with dictionary interfaces as long as they use `string`, `enum`, or `integer` (`int`, `short`, `byte`, and so on) keys:

- `IDictionary<,>`
- `IReadOnlyDictionary<,>`

WARNING You can't bind dictionaries that use non-`string` or non-integer keys, such as custom classes or `double`. For examples of binding collection types, see the associated source code for this book.

Clearly, there are quite a few nuances here, but if you stick to the simple cases from the preceding example, you'll be fine. Be sure to check for typos in your JSON files! You could also consider using explicit options validation, as described in this post: http://mng.bz/jPjr.

TIP The options pattern is most commonly used to bind POCO classes to configuration, but you can also configure your strongly typed settings classes in code by providing a lambda to the `Configure` function, as in `services.Configure<TestOptions>(opt => opt.Value = true)`.

The Options pattern is used throughout ASP.NET Core, but not everyone is a fan. In section 10.3.4 you'll see how to use strongly typed settings and the configuration binder without the Options pattern.

10.3.4 Binding strongly typed settings without the IOptions interface